

Onboarding

Welcome. This is the day-one guide: what the product is, how to get set up, how we work, and the few things that will bite you if nobody tells you about them.

Read this first, then `ARCHITECTURE.md`, then `TASKS_PHASE1.md`.

1. What xspend is

A personal finance analytics web app. Users get their transactions in, and the app categorizes them, deduplicates them, and turns them into a dashboard, insights, and projects.

Transactions arrive by **manual statement upload** today, and by **Plaid bank connection** in future. Manual upload is a deliberate product decision, not a fallback: a meaningful share of the US market banks with credit unions and small institutions, and many of those people will not connect their bank credentials to a third party. Most competing apps are Plaid-only, which excludes exactly them.

It is live, with real beta users and real financial data. Built solo.

Links

Live app

<https://xspend.vercel.app>

Backend API

<https://xspend-io.onrender.com>

Repo

<https://github.com/xspend/xspend.io> (private)

Board

GitHub Projects: **Phase 1**

What's built

Accounts. Signup, login, onboarding, multi-user with per-user data.

Ingestion. Statement upload for PDF, CSV, XLSX, and OFX. Bank detection, then template parsing for known banks. For banks with no template, an LLM fallback extracts the transactions, chunking large statements and reconciling the result against the statement's own printed balances.

Data. Transaction deduplication via a parse-stable fingerprint plus a fuzzy check. Auto-categorization. Fixed-vs-variable classification with recurrence detection. Credit card statement credits and rewards handled as contra-expenses rather than income.

Product surfaces. Dashboard, transactions list, five templated insight prompts (deterministic, no LLM, capped per user per month), projects, settings.

Banks in production today, by volume: Capital One, Chase, American Express, Bank of America, Apple Card. Capital One is the largest and has **no template**, every one of its uploads goes through the LLM fallback.

What isn't built

No automated tests. No CI. Uploads run synchronously and time out on large statements even though they succeed. There is a known, unfixed cross-user data leak. Roughly fifty spent patch scripts clutter the backend directory.

`ARCHITECTURE.md` section 9 has the full list, twenty-one open issues. None of it is hidden, and it's why you're here.

2. Read this before you touch anything

The local backend points at production

`backend/.env` sets `DATABASE_URL` to the **Neon production database**. A local `uvicorn`

will therefore read and write **live user data** unless you override it.

Always run local with an explicit override:

```
bash  
  
DATABASE_URL="sqlite:///./financeai.db" uvicorn main:app --port 8000
```

This is one stray **DELETE** away from real damage to real people's financial records. It is the single most important thing on this page. Fixing the default is fair game if you want to; until then, override it every time.

Everything on **main deploys to production immediately**

There is no CI and no staging. Push to **main**, and Render and Vercel deploy it to live users within a couple of minutes. That is why all work goes through pull requests, and why nothing merges without tests.

Building CI is on the roadmap. It is not in this milestone.

3. Access you should have

Ask if any of these are missing:

- ☐ GitHub repo: **github.com/xspend/xspend.io** (private)
 - ☐ GitHub Projects board: **Phase 1**
 - ☐ Upwork (for contract, hours, messages)
 - ☐ Slack or Teams channel, if we set one up
-

4. Local setup

```
bash

git clone git@github.com:xspend/xspend.io.git
cd xspend.io

# backend
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# run it, note the override
DATABASE_URL="sqlite:///./financeai.db" uvicorn main:app --port 8000
```

```
bash

# frontend, separate terminal
cd frontend
npm install
npm run dev
```

The frontend expects the backend on port 8000. See `frontend/src/lib/config.js`.

Tests don't exist yet. Creating the suite is your second task. After that:

```
bash

cd backend
pytest
```

5. How we work

Hours	20/week
GitHub Projects	Updated daily . The board is the source of truth for status.
Written check-in	Every other day : what you did, what's next, anything blocking.
Calls	Monday and Friday, 8:00 AM PDT / 8:30 PM IST . Monday plans, Friday reviews.
Day to day	Async.

6. Code conventions

Branch and PR. No direct pushes to `main`. Branch, PR, and describe what changed and why.

Tests ship with the change. Every behavioural change comes with a test. This codebase has been burned repeatedly by silent regressions; see `ARCHITECTURE.md` section 8.

Improve, don't rewrite. Some logic here looks strange and is load-bearing for reasons that aren't obvious from reading it. The sign conventions, the merchant canonicalization cascade, the reconciliation step, each exists because something broke in production. If something looks wrong, **flag it before changing it**. You may well be right, and if you are we want to know. But ask first.

Instrument before fixing. Every serious bug in this codebase so far was hidden behind an exception handler that swallowed the real error, or behind a missing log line. When

something fails inexplicably, add loud logging first. It has cracked every one of them in a single upload. Do not theorize for hours; make the code tell you.

zsh eats `!`. Inline `python3 -c "..."` breaks on `!`, including inside f-strings, because of history expansion. Write a `.py` file instead. This will waste ten minutes of your life exactly once.

7. Data handling

xspend holds real users' bank statements and financial records. This is the most sensitive part of the work.

- **Develop and test against synthetic data.** Never real user data.
- **No local copies of production data.** No exports, no screenshots, no dumps.
- **Production access is read-only**, and only for specific tasks agreed in advance.
- **Minimum necessary.** Query what the task needs. Don't browse.
- **AI coding assistants are fine.** Putting real user data into one is not, ever. Same for any third-party service.
- **Credentials are confidential.** Never commit, log, paste, or share them.
- **If something leaks or you think it might have, say so immediately.** A compromise of user financial data may create legal obligations, and we can't act on what we don't know about.

Full terms are in the NDA.

8. Scope of your first two weeks

The foundation: isolation and dedup correctness. Not features. Not Plaid.

Those come after, and they depend on this being solid, because the Plaid feed will run through the same dedup logic you're about to fix. A user who both connects Plaid and uploads a statement covering the same month gets the same charge twice, from two sources, with no shared ID. The fingerprint's merchant canonicalization is the only thing that can match those. That is why this milestone comes first.

The bug history in `ARCHITECTURE.md` section 8 is worth reading properly. Several of those bugs cost days, and every one of them taught something. You'll recognize the shapes.

9. Where to look

Doc	What it is
<code>docs/ONBOARDING.md</code>	This. Setup and how we work.
<code>docs/ARCHITECTURE.md</code>	The system. File map, data flow, core logic, thresholds, landmines, bug history, open issues.
<code>docs/TASKS_PHASE1.md</code>	The two-week plan, with the reasoning behind the sequencing.
GitHub Projects: Phase 1	Live status. Same tasks, as issues.

Questions are welcome, and asking early is better than asking late. Since we work async, the highest-value thing you can do is write down what you're uncertain about as you go, rather than saving it up.